

Exploring and Optimizing Bubble Sort

Ermi | Intermediate Development II | June 2026

1. Implementation

Both a basic and an optimized version of Bubble Sort were implemented in Python. The basic version uses two nested loops to repeatedly compare and swap adjacent elements. The optimized version adds a swapped flag that stops the algorithm early if no swaps occur during a pass, indicating the array is already sorted.

Basic Bubble Sort:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        # Each pass bubbles the next largest to its final position
        for j in range(0, n - i - 1):
            # Swap if left element is greater than right
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
```

Optimized Bubble Sort:

```
def bubble_sort_optimized(arr):
    n = len(arr)
    for i in range(n):
        swapped = False # Reset flag each pass
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped: # No swaps = already sorted, stop early
            break
    return arr
```

2. Test Cases and Outcomes

Random Array — Input: [37, 27, 28, 67, 10, 59, 54, 15] → Output: [10, 15, 27, 28, 37, 54, 59, 67]
Sorted correctly.

Already Sorted — Input: [1, 2, 3, 4, 5, 6] → Output: [1, 2, 3, 4, 5, 6]
No changes needed; optimized version exited after 1 pass.

Reverse Sorted — Input: [6, 5, 4, 3, 2, 1] → Output: [1, 2, 3, 4, 5, 6]
Worst case; both versions required all 6 passes.

All Identical — Input: [7, 7, 7, 7, 7] → Output: [7, 7, 7, 7, 7]

No swaps needed; optimized exited after 1 pass.

Empty Array — Input: [] → Output: []

Handled gracefully with no iterations.

Single Element — Input: [42] → Output: [42]

Trivially sorted; 1 pass, no swaps.

3. Optimization — Performance Comparison

The pass counts below demonstrate where the early-exit optimization provides measurable savings:

Scenario	Basic	Optimized	Note

Random Array	8	7	Saved 1 pass
Already Sorted	6	1	Saved 5 passes – best case benefit
Reverse Sorted	6	6	No difference – worst case
All Identical	5	1	Saved 4 passes
Empty Array	0	0	No difference
Single Element	1	1	No difference

4. Complexity Analysis

Time Complexity:

The basic Bubble Sort has $O(n^2)$ time complexity in both the average and worst cases. This arises from the two nested loops: the outer loop runs n times, and for each pass, the inner loop performs up to $n-1$ comparisons, yielding roughly $n*(n-1)/2$ total comparisons. In the best case (already sorted array), the optimized version detects no swaps on the first pass and exits immediately, achieving $O(n)$ time.

Space Complexity:

Bubble Sort has $O(1)$ space complexity. Sorting is performed in-place by swapping elements within the original array. No additional arrays or data structures are allocated, making it a memory-efficient in-place sorting algorithm.

Stability:

Bubble Sort is a stable sorting algorithm. Because elements are only swapped when one is strictly greater than its neighbor ($arr[j] > arr[j+1]$), equal elements are never swapped. This guarantees that elements with identical values retain their original relative order after sorting.

5. Observations and Conclusion

The optimization had the most dramatic effect on already-sorted and uniform arrays, reducing passes from n down to 1. For reverse-sorted arrays, no benefit was gained because swaps occur on every pass. This confirms that the early-exit optimization is most valuable when the input is partially or fully sorted.

Bubble Sort is best suited for small or nearly sorted datasets. For large datasets with random order, algorithms such as Merge Sort or Quick Sort ($O(n \log n)$) are significantly more efficient.